Software Requirements Specification

Dukaan Dost — Voice-First Order and Dues Tracker

Version: 1.2 (finalized)

Status: Hackathon MVP

Date: October 15, 2026

---

1. Purpose

Dukaan Dost helps a small shop owner turn a voice note or typed transcript into a customer order draft, review the extracted details, confirm the record, and track payments and outstanding dues.

AI output remains a draft until the owner confirms it. The app must flag uncertainty and arithmetic conflicts rather than guess.

Problem statement

> *Small shop owners often receive customer orders through spoken conversations, calls, and Hindi or Hinglish voice notes. Turning these informal requests into accurate records takes time, and details like item quantities, advances, and outstanding balances can be missed or misremembered. Shop owners need a simple way to convert voice or text orders into records they can review and correct, then track payments and dues with confidence.*

2. Product goals

- Reduce manual entry of customer orders.
- Support Hindi, English, and Hinglish input.
- Let owners inspect and correct extracted details.
- Track order totals, advances, later payments, and outstanding balances.
- Prepare reminders without sending them automatically.
- Provide a clear web demo of the workflow.

3. Scope

The MVP includes:

- A responsive web app for one shop owner.
- Typed transcript entry and audio recording or upload.
- Speech-to-text for supported formats.
- Order extraction, validation, editable persistent drafts, and explicit confirmation.
- Customer lookup by name and payment tracking.
- An on-demand outstanding-dues summary.
- Editable reminder drafts and a prefilled messaging link.
- Owner authentication and request limits before public deployment.

The MVP excludes automatic WhatsApp sending, payment processing, accounting or inventory integrations, invoice-photo extraction, and multiple shops or staff accounts.

Multi-order extraction from one long transcript is a future enhancement. If a note mentions multiple customers, the MVP flags it and lets the owner edit the transcript to one customer and rerun extraction.

4. Users and assumptions

The primary user is a shop owner who enters or records orders, reviews extracted details, confirms them, records payments, and checks dues.

The MVP serves one shop and one owner. The UI uses English labels; input and reminder templates may use Hindi or Hinglish. Dates use Asia/Kolkata time. Demo data is synthetic. The customer's preferred reminder language is separate from the language of the source note.

5. Main workflow

1. The owner enters a transcript or records/uploads a voice note.
2. The app saves a persistent draft before transcription or extraction begins.
3. Audio is transcribed; the owner may edit the transcript.
4. AI extracts order details and flags uncertainty or conflicts.
5. The owner edits the transcript or fields and confirms the draft.
6. The app saves the order and any advance payment stated in the note.
7. The owner records later payments.
8. The dashboard shows order totals and outstanding balances.
9. The owner creates and edits a reminder, then copies it or opens a messaging app with the message prefilled.

The app must not save an order or send a customer message solely because AI generated it.

6. Functional requirements

FR-1: Capture order information

The app shall accept typed transcripts and audio recordings or uploads.

Supported formats are OGG, OPUS, WEBM, M4A, MP3, and WAV, subject to the selected transcription provider's accepted formats. If the provider does not accept an uploaded format, the app shall convert it with FFmpeg or display a clear error while retaining the draft.

Acceptance criteria

- Typed transcript entry works without audio.
- The owner can record or upload a supported audio file.
- Each audio file is limited to 10 MB; the full request is limited to 12 MB.
- Unsupported or unreadable audio produces a clear error.
- The source input and draft are persisted before an AI provider is called.

FR-2: Transcribe and preserve input

The app shall transcribe supported audio and display the transcript for review.

Acceptance criteria

- The transcript is editable before extraction or confirmation.
- Provider failure does not erase the draft or transcript.
- Original audio is deleted after processing where feasible and is not written to application logs.

FR-3: Extract order details

The app shall extract, when explicitly stated:

- Customer name and phone number.

- Item descriptions, quantities, and units.
- Unit prices and line totals.
- Order total and amount already paid.
- Order date and due date.
- Source-note language.
- Customer-preferred reminder language, only when stated.
- Review flag and review notes.

Acceptance criteria

- Missing or ambiguous fields remain empty; the model must not guess.
- “Udhaar,” “baaki,” and “hisaab” are payment context only when their meaning and amounts are clear.
- An order date is not treated as a due date.
- For a due date, “kal” means tomorrow, but the app adds a review note asking the owner to confirm it.
- For a due date, “Friday” means the next upcoming Friday; if today is Friday, it means Friday one week later. The app adds a review note asking the owner to confirm it.
- Other unclear relative dates remain empty and are flagged.
- Notes naming multiple customers are flagged and not merged. The owner can edit the transcript to one customer and rerun extraction on the same draft.
- The owner can edit every extracted field.
- Source-note language is stored separately from the customer's preferred reminder language.

FR-4: Persist drafts and validate details

The app shall save each draft before calling an AI provider.

Draft statuses: RECEIVED, PROCESSING, READY, NEEDS_REVIEW, EXTRACTION_FAILED, CONFIRMED, and DISCARDED.

Acceptance criteria

- Each draft has a unique ID and can be retrieved after a page refresh.
- Provider failure leaves the draft available for retry or manual editing.
- Money uses decimal arithmetic; negative amounts are rejected.
- If all line totals are present, their sum is compared with the stated order total.
- A mismatch forces review, regardless of the model's review flag. The note shows both values, such as “Items total ₹450; stated order total ₹400.”
- A missing order total, a past due date, or an invalid phone number forces review.
- An initial payment greater than the total is flagged.
- **A missing total cannot be acknowledged away. The owner must enter a total before confirming.**
- Review acknowledgement may cover softer warnings, such as a past due date or an unclear date.

FR-5: Confirm and save orders

The app shall save an order only after explicit owner confirmation. The confirmation request shall include an **acknowledgedReview** boolean.

Acceptance criteria

- Confirmation is distinct from extraction.

- Confirmation of a NEEDS_REVIEW draft is rejected unless acknowledgedReview is true.
- Confirmation is always rejected if the order total is missing.
- Each order has a unique database constraint on its originating draft_id.
- If two confirmations race and one hits that constraint, the app returns the already-created order instead of an error.
- Only confirmed orders appear in the dues dashboard and summary.
- Confirmed orders can be cancelled. Cancellation keeps payment records and excludes the order from outstanding dues.
- The owner can discard a draft before confirmation.

FR-6: Manage customers

The app shall find or create customers by name. Customer fields are ID, name, optional phone, optional preferred reminder language, and created/updated timestamps.

The owner can edit customer details. Reminder language does not inherit from the source-note language.

FR-7: Track payments

The app shall record full or partial payments against confirmed orders.

Acceptance criteria

- If an order note states an advance, confirmation creates an initial Payment record.
- Later payments record amount, date/time, and optional note.
- Paid amount is derived from payment records; outstanding is total minus payments.
- Payments must be greater than zero and cannot exceed the outstanding balance.
- Orders display UNPAID, PARTIAL, or PAID status.
- Cancelled orders do not accept new payments; existing payments remain on record.

FR-8: Show orders and outstanding dues

The app shall provide an on-demand dashboard for confirmed orders. It displays customer, order date, total, paid amount, outstanding amount, and overdue status. Drafts, cancelled orders, and fully paid orders are excluded from outstanding dues. The summary is generated when opened or refreshed; no scheduled job is required. Dates use Asia/Kolkata time.

FR-9: Prepare reminders

The app shall create editable reminder drafts using the customer's preferred language when available.

The app does not send reminders automatically. The owner may copy the text or open a prefilled WhatsApp link when a valid phone number exists. Opening a messaging link is not proof that a message was sent.

FR-10: Protect access and control usage

Before public deployment:

- All endpoints require owner authentication.
- The password comes from deployment configuration, not source control.
- Failed login attempts are throttled.
- AI extraction is limited to 10 requests per minute per client by default.
- Transcript length is limited to 20,000 characters.

- Audio limits follow FR-1.
- Behind a trusted reverse proxy, Spring shall use forwarded headers for client IP detection. Only configured trusted proxies may supply those headers.
- Provider failures and rate limits preserve drafts.
- Demo data contains no real customer information.

7. Special draft outcomes

Previous balance: A note such as “Gupta store ka 800 rupaye baaki hai” creates a NEEDS_REVIEW draft. The owner may confirm it as a single “Previous balance” item for ₹800. It counts as an outstanding balance and is visibly labelled as a previous balance, not a new sale.

Multiple customers: The app flags the draft and does not merge customers. The owner edits the transcript to one customer and reruns extraction.

8. Data model

- **Customer:** ID, name, phone, preferred reminder language, timestamps.
- **OrderDraft:** ID, status, source type, transcript, temporary audio reference, extraction JSON, review flag/notes, timestamps.
- **Order:** ID, unique originating draft reference, customer reference, entry type (ORDER or PREVIOUS_BALANCE), order/due dates, currency, total, status, transcript, extraction JSON, timestamps.
- **OrderItem:** ID, order reference, description, quantity, unit, unit price, line total.
- **Payment:** ID, order reference, positive amount, payment date/time, optional note.
- **ReminderDraft:** ID, order reference, language, text, status, timestamp.

9. Non-functional requirements

- The main workflow works on mobile and desktop browsers.
- Normal screen actions complete within two seconds, excluding provider processing.
- The owner can inspect the transcript, extracted fields, and warnings before confirmation.
- Provider failure preserves the draft.
- Uploaded-file access is restricted; audio is deleted after processing where feasible.
- Provider credentials are not exposed in client code.
- Public deployments use TLS.
- In-memory rate limiting is acceptable for a single instance; multiple instances require a shared rate-limit store.
- UI controls have labels, keyboard support, and readable contrast.

10. Technical and deployment requirements

- **Backend:** Spring Boot REST API.
- **Frontend:** responsive web interface.
- **Database:** PostgreSQL.
- **Extraction:** Spring AI structured output with server-side validation.
- **Transcription:** configurable provider; verify accepted formats and convert unsupported formats with FFmpeg.

- **Database migrations:** Flyway. Hibernate shall use schema validation, not automatic schema updates.
- **Container image:** include FFmpeg if audio conversion is enabled.
- **Messaging:** editable draft and prefilled WhatsApp link only.

11. Acceptance transcripts

1. “Sharma ji ko 2 carton tel dena, total 2400 rupaye. 500 advance de diya, baaki Friday ko.”
   Expected: ₹2,400 total, ₹500 advance, Friday resolved as the next upcoming Friday with a review note.
2. “Ramesh ke liye chawal aur dal bhejna, hisaab baad mein kar lenge.”
   Expected: NEEDS_REVIEW; no invented total; confirmation blocked until the owner enters a total.
3. “Gupta store ka 800 rupaye baaki hai, kal tak de denge.”
   Expected: NEEDS_REVIEW; owner can confirm a “Previous balance” item for ₹800; “kal” means tomorrow and requires confirmation.
4. “Meena ko 5 kilo atta 250 ka aur Salim bhai ko 2 dabbe chai 320 ke; Meena ne 100 de diye.”
   Expected: multiple-customer warning; no merged order. Owner can edit the transcript to one customer and rerun extraction.
5. “Verma ji ke 3 packet cheeni 150 each, total 400 likh do.”
   Expected: review note shows items total ₹450 and stated total ₹400.

12. Solo-builder build order

1. Flyway schema, entities, and repositories.
2. Validation service and deterministic unit tests.
3. Extraction DTO and service; exercise the five transcripts against the model when credentials are configured.
4. Draft and confirmation endpoints, then payment handling.
5. Capture/review UI, then transcription.
6. Dashboard, reminders, authentication, limits, and deployment.

Authentication and limits must be in place before public deployment. Cancellation, customer editing, and UI polish follow the core workflow.

13. MVP completion criteria

The MVP is ready to demonstrate when the owner can submit or upload a note, retrieve its draft after refresh or provider failure, review and correct extraction, confirm without duplicates, record an advance and a later payment, see the correct outstanding balance, and observe the five acceptance cases handled without guessing.

14. Pitch alignment

Describe the MVP as: “Voice or text order in, reviewed customer order out, with payments and dues tracked.”

The trust feature is owner review before saving. Do not claim WhatsApp automation or automatic sending. Multi-order extraction is a planned enhancement unless implemented.